Use Case Description for Database Management

Use Case 1

- **Use case**: Create User Record
- **Primary actor**: Authentication System Actor
- **Goal in context**: To create a new user entry in the database so that the system can authenticate users.
- **Preconditions**:
    - The database is online and accessible.
- **Trigger**: A new user account needs to be created.
- **Scenario**
    1. The Authentication System Actor sends a request to create a new user record.
    2. The system validates the provided user information.
    3. The system stores the new user record in the database.
    4. The system triggers the Save Hashed Passwords use case to securely store the user's password.
    5. The system confirms successful creation of the user record.
- **Postcondition**
    - A new user record exists in the database.
    - The user's hashed password is securely stored.
- **Exceptions**
    1. The user data is invalid. The system rejects the request and returns an error message.
- **Priority**: High.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 2

- **Use case**: Save Hashed Passwords
- **Primary actor**: Authentication System Actor
- **Goal in context**: Store a hashed version of a user's password so that the system can authenticate users without storing plaintext passwords.
- **Preconditions**:
    - A user record is being created.
    - The Authentication System Actor provides a password to be stored.
    - The system has access to a hashing mechanism.
- **Trigger**: The Create User Record use case requires the system to store a hashed password.
- **Scenario**
    1. The Authentication System Actor provides the plaintext password to the system.
    2. The system applies a hashing algorithm to the password.
    3. The system stores the hashed password in the database.
    4. The system determines that the credentials are valid.
    5. The system returns a message indicating that the hashed password has been saved successfully.
- **Postcondition**
    - The user's hashed password is stored in the database.
- **Exceptions**
    - None
- **Priority**: High.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Internal system call
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 3

- **Use case**: Validate Credentials
- **Primary actor**: Authentication System Actor
- **Goal in context**: To verify a user's identity by checking provided credentials against stored credential data.
- **Preconditions**:
    - User records already exist in the database.
- **Trigger**: A user attempts to log in.
- **Scenario**
    1. The Authentication System Actor sends the user's credentials for validation.
    2. The system activates the Retrieve Credentials use case to obtain stored credential data.
    3. The system compares the provided credentials with the stored hashed passwords.
    4. The system determines that the credentials are valid.
    5. The system returns a successful authentication message.
- **Postcondition**
    - The system confirms the user's identity.
    - A valid authentication response is returned
- **Exceptions**
    - Invalid credentials. The system returns a failed authentication message.
    - User record does not exist. The system returns a message indicating the record does not exist.
- **Priority**: High.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 4

- **Use case**: Retrieve Credentials
- **Primary actor**: Authentication System Actor
- **Goal in context**: Retrieve stored credential data for a user so that the system can validate credentials.
- **Preconditions**:
    - The user record exists in the database.
- **Trigger**: The Validate Credentials use case requires stored credentials to compare submitted credentials.
- **Scenario**
    1. The Authentication System Actor requests credential data for a specific user.
    2. The system locates the user record in the database.
    3. The system retrieves the stored credentials.
    4. The system returns the retrieved credential data to the Authentication System Actor.
- **Postcondition**
    - The system provides the stored credentials needed for authentication.
- **Exceptions**
    - User record does not exist. The system returns a message indicating the record does not exist.
- **Priority**: High.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Internal system call
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 5

- **Use case**: Create Match Record
- **Primary actor**: Game Registry Actor
- **Goal in context**: Create a new match record in the database so the system can store and manage all data related to that match.
- **Preconditions**:
    - Required match information is available.
- **Trigger**: A new match is initiated and must be recorded in the system.
- **Scenario**
    1. The Game Registry Actor sends a request to create a new match record.
    2. The system validates the provided match information.
    3. The system generates a new match entry in the database.
    4. The system invokes the Store Game Data use case to store players' moves, game state, and game result.
    5. The system returns a message indicating that the match record has been created.
- **Postcondition**
    - A match record is created and stored in the database.
- **Exceptions**
    - The match data is incomplete or invalid. The system rejects the request and returns a message.
- **Priority**: High.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 6

- **Use case**: Store Game Data
- **Primary actor**: Game Registry Actor
- **Goal in context**: Save all game data (players' moves, game state, and game result) into the database.
- **Preconditions**:
    - A match record exists.
- **Trigger**: The Create Match Record or Update Game Data use case requires game data to be stored.
- **Scenario**
    1. The Game Registry Actor sends a request to store game data for a match.
    2. The system checks if the match record exists or is valid.
    3. The system stores players' moves using the Store Players' Moves use case.
    4. The system stores the game result or game state using the Store Game Result or State use case.
    5. The system returns a message to the Game Registry Actor indicating all game data has been successfully saved.
- **Postcondition**
    - All relevant game data is stored.
- **Exceptions**
    - The match record does not exist. The system returns an error message.
- **Priority**: High.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 7

- **Use case**: Store Game Result
- **Primary actor**: Game Registry Actor
- **Goal in context**: Save the final outcome of a match so the system can determine winners, and track game history.
- **Preconditions**:
    - A valid match record exists in the database.
    - The final game result is available.
- **Trigger**: The match comes to an end.
- **Scenario**
    1. The Game Registry Actor submits the valid final game result to the system.
    2. The system writes the game result into the database.
    3. The system confirms that the result has been successfully stored.
- **Postcondition**
    - The final game result is stored and linked to the match record.
- **Exceptions**
    - The match record does not exist. The system returns an error message.
- **Priority**: Medium.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 8

- **Use case**: Store Game State
- **Primary actor**: Game Registry Actor
- **Goal in context**: Save the current state of an ongoing match so the system can track progress.
- **Preconditions**:
    - A valid match record exists in the database.
    - The game state data is available.
- **Trigger**: The match is in progress.
- **Scenario**
    1. The Game Registry Actor submits the valid current game state to the system.
    2. The system stores the game state in the database under the match record.
    3. The system confirms that the state has been successfully saved.
- **Postcondition**
    - The game state is stored and linked to the match record.
- **Exceptions**
    - The match record does not exist. The system returns an error message.
- **Priority**: Medium.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 9

- **Use case**: Update Game Data
- **Primary actor**: Game Registry Actor
- **Goal in context**: Modify existing game information so the match record is up to date.
- **Preconditions**:
    - A valid match record exists in the database.
    - Updated game data is available.
- **Trigger**: The Game Registry Actor needs to update previously stored game information.
- **Scenario**
    1. The Game Registry Actor submits the valid updated game data for a specific match.
    2. The system invokes the Store Game Data use case to save the updated game data.
    3. The system overwrites the previously stored information in the database.
    4. The system confirms that the game data has been successfully updated.
- **Postcondition**
    - The previously stored information is overwritten.
    - The updated game data is stored under the match record.
- **Exceptions**
    - The match record does not exist. The system returns an error message.
- **Priority**: High.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 10

- **Use case**: View Game Data
- **Primary actor**: Game Registry Actor
- **Goal in context**: Access stored game information so the actor can review match outcomes and calculate the total wins or loses.
- **Preconditions**:
    - A valid match record exists in the database.
- **Trigger**: The Game Registry Actor requests to view data for a specific match.
- **Scenario**
    1. The Game Registry Actor requests to view game data for a match.
    2. The system checks if the match record exists.
    3. The system invokes the Retrieve Game Data use case to get the stored game data.
    4. The system returns the game data to the Game Registry Actor.
- **Postcondition**
    - The actor receives the requested game data.
- **Exceptions**
    - The match record does not exist. The system returns an error message.
- **Priority**: Medium.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 11

- **Use case**: Retrieve Game Data
- **Primary actor**: Game Registry Actor
- **Goal in context**: Retrieve stored game information so the system can present it to the actor.
- **Preconditions**:
    - A valid match record exists in the database.
- **Trigger**: The View Game Data use case requires stored game data.
- **Scenario**
    1. The system receives a request to retrieve game data for a specific match.
    2. The system checks if the match record exists.
    3. The system retrieves the game result using the Retrieve Game Result use case (if available).
    4. The system retrieves the game state using the Retrieve Game State use case (if available).
    5. The system returns the game data to the View Game Data use case.
- **Postcondition**
    - The system provides the requested game data.
- **Exceptions**
    - The match record does not exist. The system returns an error message.
- **Priority**: Medium.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Internal system call
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 12

- **Use case**: Retrieve Game Result
- **Primary actor**: Game Registry Actor
- **Goal in context**: Retrieve the outcome of a match so the system can display the winner or final score.
- **Preconditions**:
    - A valid match record exists in the database.
    - A game result for a match has been stored.
- **Trigger**: The Retrieve Game Data use case requires the game result.
- **Scenario**
    1. The system locates the match record in the database if it exists.
    2. The system retrieves the stored game result.
    3. The system returns the game result to the Retrieve Game Data use case.
- **Postcondition**
    - The system provides the stored game result.
- **Exceptions**
    - The match record does not exist. The system returns an error message.
- **Priority**: Low.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Internal system call
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 13

- **Use case**: Retrieve Game State
- **Primary actor**: Game Registry Actor
- **Goal in context**: Retrieve the stored game state so the system can show the current status of the match.
- **Preconditions**:
    - A valid match record exists in the database.
    - Game state data has been stored.
- **Trigger**: The Retrieve Game Data use case requires the game state.
- **Scenario**
    1. The system locates the match record in the database if it exists.
    2. The system retrieves the stored game state.
    3. The system returns the game state to the Retrieve Game Data use case.
- **Postcondition**
    - The system provides the stored game state.
- **Exceptions**
    - The match record does not exist. The system returns an error message.
- **Priority**: Low.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Internal system call
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 14

- **Use case**: Update Leaderboard
- **Primary actor**: Leaderboard System Actor
- **Goal in context**: Refresh the leaderboard so that the ranking is up to date.
- **Preconditions**:
    - Player records exist in the database.
- **Trigger**: The Leaderboard System Actor requests a leaderboard update.
- **Scenario**
    1. The Leaderboard System Actor requests a leaderboard update.
    2. The system invokes the Retrieve Players' Records use case to gather the players' information.
    3. The system recalculates the rankings based on players' records.
    4. The system updates the leaderboard in the database.
    5. The system returns a message indicating that the leaderboard has been updated.
- **Postcondition**
    - The leaderboard shows the up-to-date ranking.
- **Exceptions**
    - Player records are unable to retrieved. The system returns an error message.
- **Priority**: High.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 15

- **Use case**: Retrieve Players' Records
- **Primary actor**: Leaderboard System Actor
- **Goal in context**: Retrieve stored players' records so the system can update the leaderboard.
- **Preconditions**:
    - Players' records exist in the database.
- **Trigger**: The Update Leaderboard use case requires players' records.
- **Scenario**
    1. The system receives a request to retrieve players' records.
    2. The system retrieves players' information from the database.
    3. The system sends data to the Update Leaderboard use case.
- **Postcondition**
    - The system provides the players' records needed for updating the leaderboard.
- **Exceptions**
    - Player records are unable to retrieved. The system returns an error message.
- **Priority**: Medium.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Internal system call
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 16

- **Use case**: Store Players' Record
- **Primary actor**: Leaderboard System Actor
- **Goal in context**: Save a player's record into the database so it can be used for leaderboard updates.
- **Preconditions**:
    - Player's data is available to store.
- **Trigger**: A match has concluded and the system needs to store updated player's data.
- **Scenario**
    1. The Leaderboard System Actor requests and submits data to store into a player's record.
    2. The system validates the submitted data.
    3. The system stores the player's record in the database.
    4. The system may invoke the Update Players' Records use case if the record already exists and requires modification.
    5. The system returns a message indicating that the record has been successfully stored.
- **Postcondition**
    - The player's record is stored or updated in the database.
- **Exceptions**
    - Invalid record data. The system rejects the request and returns an error message.
- **Priority**: Medium.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service Request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 17

- **Use case**: Update Players' Record
- **Primary actor**: Leaderboard System Actor
- **Goal in context**: Modify an existing player's record so that the data is up to date.
- **Preconditions**:
    - A player record already exists in the database.
    - Updated performance data is valid.
- **Trigger**: The Store Players' Record use case determines if the record already exists in the database.
- **Scenario**
    1. The system identifies that the player's record already exists and submits the new data.
    2. The system validates the submitted data.
    3. The system updates the existing player record in the database.
    4. The system returns a message indicating that the record has been successfully updated.
- **Postcondition**
    - The player's record is updated.
- **Exceptions**
    - Invalid record data. The system rejects the request and returns an error message.
- **Priority**: Medium.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Internal system call
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 18

- **Use case**: Modify Database
- **Primary actor**: System Administrator
- **Goal in context**: Change existing database content or structure so the system remains accurate and organized.
- **Preconditions**:
    - The System Administrator is authenticated and authorized to modify database.
- **Trigger**: The System Administrator sends a request to modify the database.
- **Scenario**
    1. The System Administrator requests to modify database content.
    2. The system locates the data.
    3. The system validates the submitted modified data.
    4. The system may invoke the Delete Data use case if the modification requires removing some data.
    5. The system returns a message indicating that the modification has been successfully applied.
- **Postcondition**
    - The database is updated with the modified data.
- **Exceptions**
    - Invalid modified data. The system rejects the request and returns an error message.
- **Priority**: High.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 19

- **Use case**: Delete Data
- **Primary actor**: System Administrator
- **Goal in context**: Remove outdated or unnecessary data from the database so that the system remains well-defined.
- **Preconditions**:
    - The System Administrator is authenticated and authorized to delete data.
    - The data to delete exists in the database.
- **Trigger**: The System Administrator sends a request to delete some data in the database.
- **Scenario**
    1. The System Administrator requests to delete data in the database.
    2. The system locates the data to delete.
    3. The system deletes the data from the database.
    4. The system returns a message indicating that the data has been successfully deleted.
- **Postcondition**
    - The targeted data is deleted from the database.
- **Exceptions**
    - The targeted data does not exist. The system returns an error message.
- **Priority**: Medium.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None

Use Case 20

- **Use case**: View Database
- **Primary actor**: System Administrator
- **Goal in context**: Access and inspect the database so that the system remains accurate and organized.
- **Preconditions**:
    - The system administrator is authenticated and authorized to view the database.
    - The database is accessible
- **Trigger**: The System Administrator sends a request to view specific data in the database.
- **Scenario**
    1. The System Administrator requests to view specific database content.
    2. The system retrieves the requested data from the database.
    3. The system provides the data to the System Administrator.
- **Postcondition**
    - The System Administrator receives the requested database information.
- **Exceptions**
    - The targeted data does not exist. The system returns an error message.
- **Priority**: High.
- **When Available**: N/A
- **Frequency of Use**: N/A
- **Channel to Actor**: Service request
- **Secondary Actors**: None
- **Channels to Secondary Actors**: N/A
- **Open Issues:** None